DOCUMENTACIÓN DE PROPUESTA TÉCNICA DEL SOFTWARE

PROYECTO:
SOFTWARE PARA LA CREACIÓN DE LA APLICACIÓN “ALERTA MUJER”

AUTOR: FREINIER STEVEN CARDONA PEREZ

SERVICIO NACIONAL DE APRENDIZAJE –SENA
ANALISIS Y DESARROLLO
DE SOFTWARE – 3145555

2026

Resumen

Esta propuesta explica por qué Alerta Mujer se organiza en una aplicación móvil Android, una interfaz web, un servicio central y una base de datos. Relaciona las tecnologías elegidas con necesidades concretas: activar una alerta con rapidez, limitar el acceso a datos sensibles, conservar el contexto de una emergencia y permitir la atención administrativa. El documento presenta los beneficios y las condiciones de cada decisión, sin convertir una característica técnica en una promesa de respuesta inmediata. La solución integra React Native con Expo Router para móvil, React con Vite para web, Java 21 con Spring Boot para el servicio y PostgreSQL con Liquibase para los datos. Las operaciones habituales usan REST; el chat utiliza WebSocket con STOMP por medio del servidor. La propuesta incluye criterios de configuración, riesgos y aceptación que permiten evaluar la solución como producto académico y social funcional.

Palabras clave: Alerta Mujer, software, documentación de producto, seguridad, apoyo.

Contenido

1. Finalidad de la propuesta y criterios de decisión
2. Necesidades que gobiernan la arquitectura
3. Estructura general de la solución
4. Aplicación móvil con React Native y Expo Router
5. Interfaz web con React y Vite
6. Servicio central con Java y Spring Boot
7. Datos estructurados con PostgreSQL y Liquibase
8. Operaciones REST para los procesos comunes
9. Chat mediante WebSocket y STOMP
10. Identidad verificación y sesiones
11. Relaciones de confianza entre cuentas
12. Gestión de emergencias y ubicación
13. Evidencias y almacenamiento privado
14. Configuración de ambientes y ejecución local
15. Seguridad privacidad y auditoría
16. Operación mantenimiento y crecimiento
17. Riesgos y medidas de tratamiento
18. Criterios de aceptación y evolución propuesta

1 Finalidad de la propuesta y criterios de decisión

La propuesta técnica traduce necesidades del proyecto en decisiones de construcción y operación. Alerta Mujer debe permitir que una persona registrada active una emergencia, que sus contactos aceptados consulten la alerta y que personal autorizado la atienda desde la web. También debe proteger datos personales, coordenadas, imágenes y conversaciones. Estas tareas exigen una arquitectura clara, un comportamiento comprensible para el usuario y una forma de mantener el sistema sin alterar la información ya registrada. La propuesta explica cómo las tecnologías seleccionadas ayudan a cumplir ese propósito y dónde aparecen sus límites.

Los criterios de elección son compatibilidad con Android y navegadores actuales, separación de responsabilidades, control de acceso, persistencia confiable, posibilidad de pruebas y continuidad de mantenimiento. El costo de adoptar y operar cada herramienta también importa, aunque este documento no presenta un presupuesto financiero que no haya sido medido. La experiencia de la usuaria ocupa un lugar central: el rendimiento de una operación no se valora solo por el tiempo de respuesta del servidor, sino por la claridad con que se confirma o se explica un resultado. Las decisiones se apoyan en los requisitos y en la arquitectura descrita por el proyecto; su aceptación requiere verificar los flujos reales de extremo a extremo (Cardona Pérez, 2025, 2026).

La decisión técnica también se juzga por lo que hace comprensible el producto. Una tecnología puede ser adecuada para el equipo y, aun así, necesitar una explicación más clara para quien recibe una alerta. Por ello, cada componente se vincula con un resultado observable: registro confirmado, vínculo aceptado, caso visible o mensaje recibido. Los criterios no incluyen beneficios atribuidos a una herramienta sin medición; la propuesta conserva las hipótesis de rendimiento como asuntos de prueba.

2 Necesidades que gobiernan la arquitectura

La solución necesita atender al menos tres contextos distintos. En móvil, la persona gestiona su cuenta y contactos, activa la emergencia y puede aportar ubicación, fotografías y mensajes. En la web pública, un visitante encuentra información y rutas de acceso sin ver datos protegidos. En el panel web, una persona administradora consulta y atiende casos, administra cuentas y revisa registros autorizados. Un servicio común debe aplicar las mismas reglas aunque las pantallas sean diferentes. La base de datos conserva las relaciones, estados e historial necesarios para reconstruir lo que el sistema recibió.

Esta distribución responde al riesgo de tomar decisiones inconsistentes en clientes separados. Si una aplicación decidiera por sí sola quién puede ver una fotografía, el otro cliente podría aplicar una regla distinta. Al concentrar las comprobaciones en el servidor, la interfaz se dedica a orientar a la persona y el servicio confirma la autorización. La arquitectura también reconoce que la red, el GPS y los permisos pueden fallar. Por ello, cada cliente necesita representar acciones pendientes, errores y la última información confirmada. El diseño debe sostener esa precisión en los flujos de alerta, no solamente en pantallas administrativas de uso ordinario (Cardona Pérez, 2025).

Una consecuencia de esta organización es que los datos deben conservar el mismo significado al pasar entre componentes. “Activa”, “en atención” y “finalizada” no pueden representar situaciones distintas en móvil y web. La fecha de una ubicación tampoco puede desaparecer al cruzar la interfaz. La propuesta requiere contratos de comunicación y pruebas de integración para detectar estas divergencias. Es una medida técnica con efecto directo en la comprensión de quienes observan una emergencia.

3 Estructura general de la solución

Alerta Mujer sigue una separación por capas: interfaces para las personas, lógica central para reglas y seguridad, y persistencia para datos estructurados. El servidor se organiza como un monolito modular. Esto significa que las funciones de identidad, contactos, emergencias, evidencias, chat y administración tienen responsabilidades diferenciadas dentro de una sola aplicación de servicio. La opción facilita compartir reglas y operar un proceso central sin imponer coordinación entre múltiples servicios independientes. También exige disciplina para conservar los límites entre módulos y evitar dependencias innecesarias.

Los clientes móvil y web se comunican con el servicio mediante operaciones autenticadas. El servidor valida solicitudes, aplica permisos y decide qué información devuelve. PostgreSQL guarda datos de cuentas, relaciones y casos; las imágenes de evidencia se conservan en almacenamiento privado, asociadas a registros que permiten consultarlas bajo autorización. Esta estructura permite explicar el recorrido de una alerta de forma sencilla: el teléfono solicita, el servicio comprueba y registra, y los actores autorizados reciben o consultan el resultado. La separación no garantiza disponibilidad por sí misma. Para funcionar correctamente necesita configuración coherente de direcciones, credenciales, conectividad y almacenamiento en cada ambiente (Cardona Pérez, 2026).

El monolito modular resulta razonable mientras el equipo puede mantener una separación interna legible y una operación central. Esta decisión no impide evolucionar el sistema, pero evita introducir distribución antes de justificar su costo. El seguimiento del caso puede diagnosticarse recorriendo cliente, servicio y datos, con responsabilidades definidas en cada paso. Si un módulo empieza a depender de información privada de otro sin una regla clara, conviene revisar el límite antes de ampliar funciones.

4 Aplicación móvil con React Native y Expo Router

React Native permite construir la interfaz móvil con un enfoque de componentes reutilizables y acceso a capacidades del dispositivo por medio del entorno de Expo. Expo Router organiza las pantallas y sus recorridos, lo que ayuda a mantener identificables pasos como registro, contactos, alerta activa e historial. La elección responde a la necesidad de una aplicación Android con navegación consistente y capacidad de solicitar ubicación y cámara. Los mensajes visibles deben explicar permisos denegados y estados de envío en palabras sencillas, porque la persona puede usarla en una situación de tensión.

La ventaja de compartir componentes y reglas de presentación no elimina las condiciones propias del teléfono. La ubicación puede no estar disponible, la cámara depende de permisos y las tareas en segundo plano requieren pruebas específicas del sistema operativo y del dispositivo. Por ello, la propuesta limita las afirmaciones de funcionamiento a comportamientos comprobados en el ambiente evaluado. La instalación y las pruebas deben verificar qué funciones se conservan cuando se suspende la aplicación o cambia la conexión. React Native y Expo facilitan el desarrollo y la iteración, pero la aceptación del producto depende de que las operaciones sensibles se confirmen también en el servidor y se comuniquen correctamente a la usuaria (Cardona Pérez, 2025, 2026).

En la aplicación móvil, la navegación debe sostener el regreso seguro desde pantallas secundarias al estado de la emergencia. Una foto que se está cargando, una sesión vencida o una solicitud de permiso pueden interrumpir el recorrido esperado. Los componentes reutilizables ayudan a mantener mensajes iguales, pero se deben probar en teléfonos reales con diferentes tamaños de pantalla. La aceptación incluye comprobar que los controles principales permanezcan reconocibles con preferencias de idioma y apariencia disponibles.

5 Interfaz web con React y Vite

La aplicación web reúne dos experiencias que comparten una base técnica y tienen permisos diferentes. El visitante accede a información general sobre Alerta Mujer; el administrador ingresa con credenciales para revisar y gestionar casos. React facilita dividir la interfaz en componentes con responsabilidades visibles, como listados, detalles, formularios y conversación. Vite proporciona un entorno de desarrollo y construcción adecuado para una aplicación web moderna. La elección ayuda a mantener coherencia de navegación y mensajes sin mezclar contenido público con datos protegidos.

La separación debe comprobarse más allá de las rutas de pantalla. Una persona no autorizada no debe obtener información de emergencias mediante solicitudes directas al servicio, aunque conozca una dirección de la web. Del mismo modo, el panel debe presentar el estado confirmado por el servidor al iniciar la atención o cambiar una cuenta. La interfaz web puede mejorar la lectura de datos complejos, pero debe conservar fechas, estados y límites de interpretación: un punto de ubicación representa la última lectura recibida, no una presencia continua. Los mensajes para visitantes deben describir el propósito real del producto y evitar prometer atención que depende de participación humana y conectividad (Cardona Pérez, 2025, 2026).

Para administración, la web debe hacer visible qué información procede del caso y cuál es una acción del operador. Una tabla de emergencias sirve para localizar; el detalle sirve para decidir con contexto; el historial permite revisar. Mezclar esas tareas en un mismo indicador puede ocultar cambios de estado. El visitante requiere un recorrido más breve y sin credenciales. La arquitectura web debe conservar esas diferencias incluso si ambas experiencias comparten la misma tecnología de presentación.

6 Servicio central con Java y Spring Boot

El servicio central utiliza Java 21 y Spring Boot 4.1.1 para reunir las operaciones de negocio. Su función es recibir solicitudes de móvil y web, validar datos, autenticar sesiones, aplicar reglas y conservar el resultado en los sistemas de almacenamiento. Al situar estas decisiones en un mismo servicio se reduce el riesgo de que cada cliente interprete de manera diferente quién puede activar, consultar o cerrar una emergencia. La organización modular permite que identidad, contactos, casos, imágenes y chat evolucionen con responsabilidades distinguibles.

La propuesta favorece esta estructura porque resulta comprensible para un equipo académico que necesita explicar, probar y mantener el producto. Un servicio central facilita seguir el recorrido de una operación y observar errores en un solo punto de aplicación. Su límite principal es que la disponibilidad de todas las funciones depende de la salud de ese servicio y de sus conexiones. También obliga a mantener pruebas de permisos y de transiciones de estado, ya que una modificación en una regla puede afectar varios clientes. La versión exacta del entorno debe fijarse para cada entrega y comprobarse junto con las dependencias, evitando que una instalación posterior cambie el comportamiento esperado (Cardona Pérez, 2026).

El servidor también es el lugar adecuado para validar invariantes, por ejemplo que una fotografía pertenezca a una emergencia existente y que una invitación aceptada corresponda a dos cuentas válidas. Estas comprobaciones no deben quedar solo en formularios, pues las solicitudes pueden llegar desde diferentes clientes. Cuando una operación falla, la respuesta del servicio debe permitir a la interfaz explicar el problema sin filtrar detalles internos. La observabilidad técnica resulta útil para investigar el fallo después.

7 Datos estructurados con PostgreSQL y Liquibase

PostgreSQL 17 conserva cuentas, vínculos entre personas, emergencias, ubicaciones, mensajes y registros asociados. Su modelo relacional permite expresar qué contacto aceptó una invitación, a qué caso pertenece una fotografía y qué cambios ocurrieron durante una atención. Esta relación importa tanto para la integridad de los datos como para el control de acceso. La base debe impedir asociaciones incoherentes y permitir consultas que devuelvan información solo a través del servicio autorizado. La estructura por áreas funcionales ayuda a comprender responsabilidades sin convertir cada dato en un sistema aislado.

Liquibase gestiona la evolución de la estructura mediante migraciones ordenadas. Una instalación puede reproducir las tablas y ajustes previstos, y una actualización puede aplicar cambios de forma controlada. Esa ventaja depende de conocer el estado de partida y ejecutar las migraciones en la secuencia definida. Cambiar una migración ya aplicada puede alterar su verificación de integridad, por lo que las correcciones deben planearse como cambios nuevos cuando corresponda. La propuesta exige además respaldos y comprobación de restauración, porque conservar datos de emergencia requiere una estrategia de recuperación. Ninguna tecnología de base de datos reemplaza por sí sola las políticas de acceso y conservación de información (Cardona Pérez, 2026).

Al diseñar cambios de datos conviene pensar en información histórica, no únicamente en la pantalla actual. Si una emergencia se finaliza, sus mensajes y evidencias deben conservar su asociación para las consultas autorizadas. Si una cuenta cambia de estado, las relaciones existentes no deben quedar ambiguas. Una migración de estructura debe probarse sobre un estado representativo y respaldado. El criterio de éxito incluye que los datos previos sigan siendo interpretables después de la actualización.

8 Operaciones REST para los procesos comunes

Las operaciones REST sirven para crear y consultar cuentas, contactos, emergencias, ubicación, evidencias e historiales. Cada solicitud representa una acción que el servidor puede validar y responder con un resultado explícito. Esta elección es adecuada para procesos en los que la aplicación necesita saber si un cambio quedó registrado, como aceptar una invitación o cerrar una alerta. También facilita separar una petición fallida de un estado confirmado: si el cliente no recibe respuesta, no debe afirmar que el servidor terminó la operación.

La ubicación de una emergencia se envía mediante solicitudes periódicas, no como una transmisión continua por el canal del chat. En la práctica, la frecuencia y la calidad de las actualizaciones dependen del dispositivo, de la conectividad y de las condiciones de ejecución. Las respuestas deben incluir información suficiente para que móvil y web presenten la última lectura confirmada con su momento. REST aporta un contrato claro entre clientes y servicio, pero no resuelve por sí solo duplicaciones, permisos o errores de red. Esas reglas deben definirse y probarse para cada operación sensible. La documentación funcional debe describir resultados esperados sin exponer detalles internos innecesarios a quienes solo necesitan usar el producto (Cardona Pérez, 2025, 2026).

El contrato de cada operación debe definir entradas válidas, respuesta de éxito, denegación de permiso y situación de error. Esto facilita pruebas y evita que el cliente deduzca resultados a partir de silencios o tiempos de espera. Para acciones repetibles, se debe examinar qué pasa si una petición se reenvía tras una desconexión. La solución puede prevenir duplicados en determinados procesos, pero esa garantía requiere verificación específica; no debe suponerse por usar HTTP o REST.

9 Chat mediante WebSocket y STOMP

La conversación durante una emergencia requiere que los mensajes lleguen con una demora percibida como breve cuando existe conexión. WebSocket mantiene un canal abierto entre cliente y servidor, y STOMP organiza el intercambio de mensajes dentro de ese canal. El servidor autentica y distribuye la conversación vinculada al caso. Por tanto, no se trata de una conexión directa entre los dispositivos de la usuaria y del administrador. El historial persiste para que una reconexión permita recuperar contexto, sujeto a los permisos de cada participante.

La configuración actual utiliza un intermediario simple en memoria. Es apropiada para una instancia de servicio, pero una operación con varias instancias exige revisar cómo se distribuyen mensajes y sesiones. La propuesta reconoce este límite antes de recomendar escalamiento. También distingue la llegada técnica de un mensaje de la lectura o respuesta humana: el canal puede estar disponible sin que un administrador conteste de inmediato. La interfaz debe indicar envío pendiente, confirmación o error y no duplicar mensajes tras reconectar. El control de acceso debe aplicarse tanto al ingreso al canal como a la consulta del historial, porque el contenido del chat puede incluir información personal delicada (Cardona Pérez, 2026).

La experiencia del chat debe coordinar dos formas de obtener datos: el canal abierto para mensajes nuevos y una consulta protegida para recuperar el historial. Esa combinación ayuda cuando el dispositivo cambia de red o reabre la conversación. Los mensajes necesitan una identidad y un orden que eviten confundir reenvíos con intervenciones nuevas. Las pruebas deben incluir cierres inesperados, sesión expirada y un intento de entrar a un caso ajeno, porque el canal en tiempo real también transmite información sensible.

10 Identidad verificación y sesiones

El registro público de la aplicación móvil utiliza una verificación mediante código de un solo uso. Esta decisión reduce la posibilidad de habilitar una cuenta sin completar la comprobación de contacto prevista en el producto. Las cuentas creadas por administración siguen otro flujo: reciben credenciales y quedan habilitadas según la operación administrativa, sin exigir el mismo paso de OTP. El servicio central debe preservar esa diferencia al autenticar y al comunicar estados a los clientes. Si el código vence o se ingresa de forma incorrecta, la aplicación necesita una respuesta que permita entender cómo continuar sin revelar información de terceros.

Las sesiones autenticadas utilizan tokens JWT. Esta tecnología transporta información verificable para identificar solicitudes, pero su uso seguro depende de configuración, duración, validación y protección de las claves. Las contraseñas se almacenan como valores derivados con BCrypt, no como texto legible. La propuesta mantiene la decisión de autorización en el servidor: una sesión válida no implica permiso para cualquier emergencia. El rol, el estado de la cuenta y la relación con el caso deben comprobarse por separado. El simulador de verificación telefónica es una capacidad de pruebas cuya configuración debe revisarse expresamente antes de usar un proveedor real; documentarlo como comprobación efectiva de un teléfono externo sería impreciso (Cardona Pérez, 2025, 2026).

El flujo de OTP depende de un servicio y de condiciones del ambiente; por ello, los errores de entrega y expiración deben tratarse como situaciones esperables. En pruebas locales, el simulador facilita ejecutar el recorrido sin un operador telefónico externo, pero no demuestra recepción real en un número. Para la cuenta administrativa interesa comprobar que la creación autorizada entrega una vía de acceso clara y que la persona puede cambiar sus credenciales

según las reglas del sistema. Ambos caminos comparten control de sesiones, no identidad de proceso.

11 Relaciones de confianza entre cuentas

El producto selecciona contactos entre personas registradas, mediante invitaciones que requieren aceptación. Esta decisión crea una relación identificable dentro del sistema y permite revocar o limitar la visibilidad según el estado del vínculo. También evita depender de una agenda externa cuyo titular podría no haber consentido el uso de sus datos. La aplicación debe distinguir la invitación enviada de la relación activa. Solo el contacto aceptado puede consultar las alertas que le corresponden; una búsqueda o un mensaje de invitación no habilitan por sí mismos el acceso.

La consecuencia técnica es que el servicio debe validar la relación en cada consulta sensible. No basta con entregar una lista filtrada por la interfaz, porque una solicitud directa podría intentar abrir otro caso. Los estados del vínculo deben tener significado estable en móvil, servidor y base de datos. La propuesta también favorece mensajes breves que expliquen por qué una persona aún no aparece como contacto activo. En el alcance actual, las alertas de estos contactos se consultan dentro de la aplicación; no se ofrece una promesa de notificaciones push del sistema operativo. Si una fase posterior incorpora otro canal de aviso, deberá revisar consentimiento, confiabilidad y reglas de privacidad antes de presentarlo como función aceptada (Cardona Pérez, 2025, 2026).

La invitación aporta una evidencia de voluntad que una lista de números guardados en el teléfono no ofrece. El modelo debe permitir consultar quién envió la solicitud, a quién se dirigió y cuál fue la decisión. La aplicación puede presentar ese estado de manera sencilla, sin exponer datos que no sean necesarios para reconocer a la persona. Al cambiar una relación, la

autorización de futuras consultas debe actualizarse en el servidor y comprobarse incluso cuando el contacto mantiene una vista abierta.

12 Gestión de emergencias y ubicación

La emergencia es el eje que reúne estados, mensajes, evidencias y posiciones recibidas. La propuesta conserva un ciclo controlado: creación por la usuaria, consulta de actores autorizados, posible atención administrativa y finalización por una acción permitida. El servicio debe validar las transiciones y devolver el estado confirmado a cada interfaz. Un cambio visual sin confirmación no es suficiente para declarar atendido o cerrado un caso. Esta regla es especialmente útil cuando varias personas consultan simultáneamente o cuando la conexión se interrumpe.

La ubicación se obtiene en el dispositivo y se envía al servicio mientras las condiciones lo permiten. El dato que puede mostrarse con certeza es la última posición recibida, junto con su hora. Una ausencia de nuevas posiciones no demuestra que la emergencia terminó ni que la persona permanezca en el mismo lugar. La propuesta exige que móvil y web expresen esa diferencia. También recomienda verificar permisos denegados, señal limitada y recuperación de conexión, porque son escenarios previsibles del uso real. La decisión de enviar coordenadas mediante operaciones REST facilita confirmar cada recepción, aunque requiere controlar solicitudes repetidas y estados de error. Ninguna de estas medidas convierte el producto en un sistema de localización garantizada (Cardona Pérez, 2025, 2026).

El estado “sin conexión” requiere un tratamiento especial: expresa que no llega actividad, no que el riesgo terminó. La propuesta recomienda que los clientes indiquen el último momento confirmado y eviten transformar automáticamente ese estado en cierre. Para pruebas, se comparan casos con ubicación inicial válida, actualización posterior y ausencia de nuevas

lecturas. El objetivo no es dibujar un recorrido imaginado, sino presentar lo recibido de una manera que ayude sin crear certeza falsa.

13 Evidencias y almacenamiento privado

Las fotografías aportan contexto a una emergencia y pueden contener información íntima de la persona o de su entorno. La ruta principal descrita en el alcance funcional es la captura iniciada por la usuaria, la conversión a WebP, la carga al servicio y el almacenamiento en un espacio privado del servidor. La solución incorpora además una posibilidad de captura automática que depende de aplicación activa y permisos. Esta capacidad debe evaluarse con criterios de consentimiento y comportamiento visible antes de incluirla como compromiso de uso. La existencia técnica de una ruta no prueba que funcione de forma continua o que responda al alcance aprobado.

La propuesta almacena el archivo fuera de la exposición pública y conserva en la base los datos necesarios para asociarlo al caso. Una consulta debe pasar por el servicio, que comprueba la autorización de la persona solicitante. El cliente debe informar si la imagen aún se prepara, si se envía, si el servidor la recibió o si la operación falló. La primera alerta no debe presentarse como portadora automática de una fotografía. Para mantener esta función, el ambiente de ejecución necesita almacenamiento persistente, control de acceso, respaldos y un procedimiento de recuperación. Comprimir la imagen mejora la transferencia; no reemplaza estas obligaciones de protección (Cardona Pérez, 2025, 2026).

La asociación entre archivo y caso debe comprobarse desde ambos extremos: al cargar y al consultar. Un archivo sin vínculo o vinculado al caso equivocado pierde valor y puede generar una exposición indebida. La retención de imágenes también requiere una decisión operativa documentada, coherente con los fines del proyecto y la normativa aplicable. Esta propuesta no

inventa un plazo de conservación. Solicita que el equipo lo defina con responsables, mecanismo de respaldo y criterio de eliminación antes de una operación sostenida.

14 Configuración de ambientes y ejecución local

La operación integrada necesita que base de datos, servicio, aplicación móvil y web compartan direcciones y parámetros coherentes. Las credenciales, claves y rutas de almacenamiento cambian según el ambiente y deben administrarse fuera del contenido público del proyecto. La propuesta establece que cada configuración tenga un propósito claro, un valor de ejemplo seguro y una comprobación antes de iniciar las pruebas. El uso de archivos de variables de entorno facilita reunir ajustes, pero no significa que todos los procesos los carguen de la misma manera. En particular, una ejecución directa de Spring requiere suministrar las variables mediante el mecanismo elegido.

Para una prueba local repetible conviene iniciar primero la base de datos, aplicar las migraciones, preparar el acceso administrativo previsto, iniciar el servicio y después abrir los clientes con las direcciones correctas. Esta secuencia permite atribuir un fallo a su componente y evita que un cliente muestre un error de red por una dependencia aún inactiva. Los manuales técnico y de instalación deben aportar los comandos exactos y las verificaciones sencillas para cada paso. La propuesta conserva aquí la regla general: una operación solo se declara disponible después de comprobar su respuesta, no por haber iniciado un proceso sin errores visibles (Cardona Pérez, 2026).

La claridad de configuración reduce errores difíciles de diagnosticar. Una dirección válida para la web en el mismo computador puede no ser accesible desde un teléfono conectado a otra red. Los manuales deben explicar esa diferencia y proponer una comprobación de conectividad antes de probar SOS o chat. Del mismo modo, un servidor que inicia no demuestra

que pueda escribir en el directorio de evidencias. Una lista breve de verificaciones posteriores al arranque protege la fidelidad de las pruebas.

15 Seguridad privacidad y auditoría

La arquitectura de seguridad combina identificación de la persona, autorización por rol y relación con el caso, protección de contraseñas y control de acceso a evidencias. Una fotografía o una ubicación no debe quedar disponible por conocer una dirección de consulta. El servidor debe validar cada solicitud y devolver únicamente la información correspondiente. La interfaz contribuye ocultando acciones improcedentes y explicando denegaciones, pero no sustituye esa comprobación. Los registros administrativos permiten revisar operaciones relevantes, siempre que eviten exponer secretos o datos personales innecesarios.

La propuesta también distingue protección en tránsito y en reposo. La comunicación pública debe configurarse con HTTPS y el canal de conversación con WSS cuando intervenga una capa segura de acceso. Un proxy puede terminar la conexión cifrada y reenviar el tráfico al servicio interno, incluida la actualización necesaria para WebSocket. El almacenamiento de imágenes exige ubicación persistente, permisos restringidos y respaldos. Las credenciales y claves no deben incluirse en versiones públicas ni en capturas de manuales. Estas decisiones reducen riesgos; su eficacia depende de configuración, revisión y pruebas de acceso indebido. El sistema debe evitar prometer seguridad absoluta o anonimato que su operación no ofrece (Cardona Pérez, 2026).

La protección debe mantenerse después de cambios ordinarios: una cuenta inhabilitada, un contacto removido o una sesión caducada ya no deberían conservar permisos por una vista que quedó abierta. Los casos de prueba de autorización pueden combinar cambios de rol y relación con solicitudes antiguas. La auditoría ayuda a reconstruir decisiones administrativas,

pero debe configurarse para no revelar mensajes completos o secretos sin necesidad. La mínima información útil suele ser mejor que una acumulación indiscriminada de datos.

16 Operación mantenimiento y crecimiento

La operación diaria requiere observar si el servicio responde, si la base de datos acepta conexiones, si las migraciones aplicadas corresponden a la versión y si las imágenes permanecen disponibles bajo autorización. Los respaldos deben verificarse mediante restauración, porque un archivo creado pero nunca recuperado ofrece una confianza engañosa. También conviene revisar registros, capacidad de almacenamiento y errores de comunicación para detectar fallos antes de que afecten la consulta de un caso. Las tareas deben asignarse a responsables concretos en el procedimiento de implementación y en el manual técnico.

El crecimiento exige una decisión específica sobre el chat. El intermediario simple en memoria funciona dentro de una instancia de servidor; si se distribuye la carga entre varias instancias, los mensajes necesitan un mecanismo compartido o una estrategia equivalente. El escalamiento de la base y del almacenamiento de imágenes requiere revisar integridad y control de acceso. La propuesta evita anticipar una capacidad de múltiples operadores o de cobertura territorial que no haya sido medida. Las mejoras se priorizan según impacto en la usuaria, protección de datos y evidencia de pruebas. Mantener documentación, migraciones y contratos de comunicación alineados resulta tan relevante como añadir nuevas pantallas (Cardona Pérez, 2026).

Una rutina de mantenimiento puede incluir revisión de estado de migraciones, capacidad disponible, respaldo recuperable y disponibilidad de las rutas que usan los clientes. Si aparece un error, la investigación debe identificar si ocurrió en el dispositivo, la red, el servicio o la persistencia. Esa separación evita atribuir a la usuaria un fallo de infraestructura. Las mejoras

deben programarse con una versión conocida y un procedimiento de retorno, pues una corrección rápida puede afectar la lectura de casos anteriores.

17 Riesgos y medidas de tratamiento

Los riesgos principales surgen de dependencias externas y de interpretaciones incorrectas del estado. Sin conectividad, una solicitud puede quedar pendiente; sin permiso de ubicación, no habrá coordenada nueva; sin persistencia de imágenes, una evidencia podría perderse; con una configuración de acceso demasiado amplia, personas no autorizadas podrían intentar consultar datos sensibles. También existe el riesgo de que una pantalla presente como confirmada una acción que el servidor rechazó. Cada caso requiere una medida observable y una prueba, no solo una recomendación general.

Riesgo	**Medida propuesta**	**Comprobación**
Conexión interrumpida	Mostrar estado pendiente y última información recibida	Repetir el flujo con red intermitente
Acceso indebido	Validar rol y vínculo en el servidor	Solicitar un caso ajeno desde cada rol
Pérdida de evidencia	Persistencia privada y respaldo verificable	Recuperar un archivo de prueba autorizado
Mensajes inconsistentes	Confirmación del servidor y recuperación del historial	Repetir envío tras reconexión

La tabla orienta la aceptación, pero no prueba que cada medida se haya ejecutado. El equipo debe registrar resultados reales y ajustar las acciones según los hallazgos. Esta disciplina evita que un riesgo sensible quede oculto detrás de una afirmación optimista (Cardona Pérez, 2025, 2026).

El tratamiento se prioriza por probabilidad y efecto sobre las personas. Un error que muestra un enlace inactivo afecta la experiencia; uno que comunica una alerta no registrada como exitosa puede afectar decisiones durante el riesgo. Por eso, la aceptación exige observar el resultado remoto y el mensaje visible de manera conjunta. Cuando una medida depende de configuración externa, la evidencia debe identificar el ambiente concreto. Así la tabla sirve para planear pruebas, no para declarar seguridad por anticipado.

18 Criterios de aceptación y evolución propuesta

La propuesta se acepta cuando las decisiones técnicas permiten completar de forma verificable los recorridos centrales: registro público con verificación, acceso administrativo, invitación aceptada, creación y cierre de una emergencia, consulta autorizada, envío de ubicación, carga manual de evidencia, conversación y revisión de historial. Cada recorrido debe incluir estados de error y comprobar que el resultado presentado coincide con el servidor. También deben validarse las restricciones de acceso a datos ajenos y la persistencia de información después de reiniciar los componentes que corresponda. Las pruebas en dispositivo físico aportan evidencia adicional sobre permisos y continuidad, distinta de la ejecución integrada local.

La evolución se debe ordenar por necesidades comprobadas. La captura automática condicionada requiere una decisión funcional explícita y pruebas de consentimiento antes de describirse como comportamiento aceptado. Las notificaciones del sistema operativo, las integraciones institucionales y la distribución de chat entre varias instancias son líneas posibles, no capacidades comprometidas por esta propuesta. Cualquier nueva función debe acompañarse de cambios en requisitos, manuales, pruebas y protección de datos. Esta regla conserva

coherencia entre lo que el producto hace, lo que el equipo puede demostrar y lo que una persona usuaria puede esperar con fundamento (Cardona Pérez, 2025, 2026).

Referencias

Cardona Pérez, F. S. (2025). *Especificación de requisitos de software de Alerta Mujer* [Documento interno del proyecto formativo, ficha 3145555]. Servicio Nacional de Aprendizaje.

Cardona Pérez, F. S. (2026). *Documentación de análisis de software de Alerta Mujer* [Documento interno de arquitectura, módulos e interfaces del proyecto formativo, ficha 3145555]. Servicio Nacional de Aprendizaje.